

6-MDP 价值迭代

马尔可夫决策过程 MDP

和上一章介绍的确定性决策过程不同，这一章介绍的是不确定的决策过程

一个马尔可夫决策过程 markov decision processes包含：

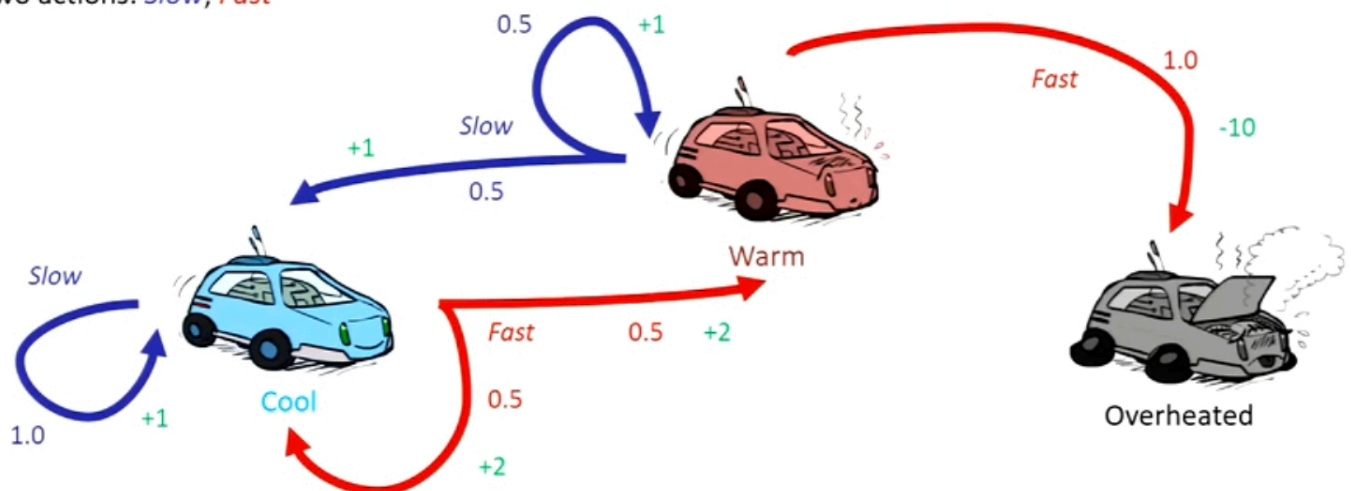
- 状态state 动作action
- 转移函数 $T(s,a,s')$ 实际上是一个条件概率， $P(s'|s,a)$ ，意味着动作的结果不是确定的
- 奖励函数 $R(s,a,s')$ 或者也可能是 $R(s)$ $R(s')$ 对**每一个**(s,a)给出评价 而所有的奖励总和为**效用值**

在确定性的搜索问题，我们要找一个plan 一个动作序列；在MDP里，需要一个策略policy π $S \rightarrow A$ ，描述不同状态下要完成什么action

最优的policy记作 π^*

一个马尔科夫链可以转换为搜索树

- A robot car wants to travel far, quickly
- Three states: **Cool**, **Warm**, Overheated
- Two actions: **Slow**, **Fast**



- 价值迭代算法
- 策略迭代算法

奖励函数和折扣 Reward func & Discount

获得奖励的先后顺序可能会有影响，所以引入一个折扣因子gamma，之后获得的奖励都乘gamma

比如gamma=0.5 奖励序列[1,2,3] = $1 + 2 \cdot 0.5 + 3 \cdot 0.25 = 2.75$ 不如[3,2,1]

▪ Given:



- Actions: East, West, and Exit (only available in exit states a, e)
- Transitions: deterministic

假设初始位置是D

gamma = 1的情况下，向左的奖励序列[0,0,0,10]，总效用值为10；向右[1]，因而向左更佳

gamma = 0.1的情况下，向左收到折扣[0,0,0,0.0001]，向右[0.1]，向右的效用值更大

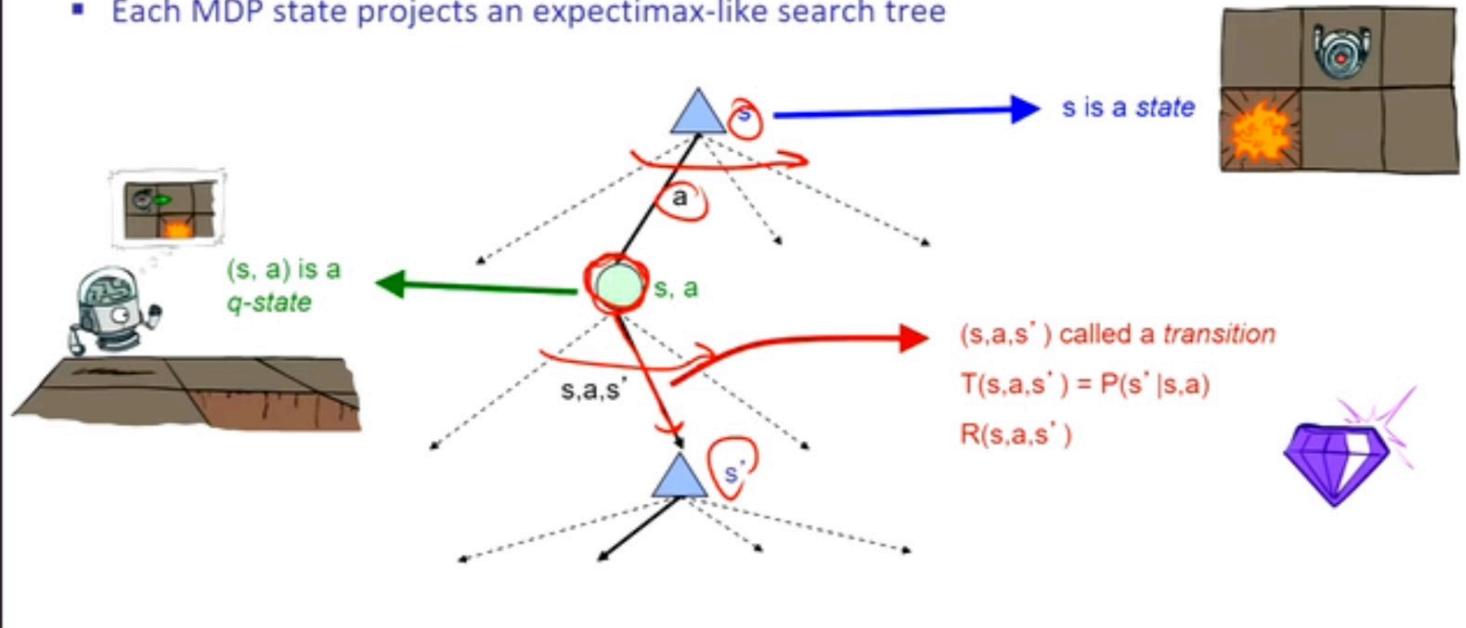
在上面的开车的例子，游戏可能会无限继续，不同的策略可能都会导致无穷大的效用值，无法判断谁更优有三种解决方法：

- 设置action的数量上限，类似深度受限搜索
- 设置discount，这样总和不会是无穷大，折扣也有助于算法收敛
- 保证对于任何策略，都会最终导致终止状态

节点价值value的计算 贝尔曼方程

MDP Search Trees

- Each MDP state projects an expectimax-like search tree



s状态的最大价值，是从这个状态开始，可能的最大效用值。用递推的方法，也就是采取所有行动后所有可能的q状态的最大值，这里的s节点相当于一个max节点

,

q状态的最大价值，对可能发生的状态s'进行加权（权重是转移函数，也就是概率），价值包含到达s'状态的奖励函数，也包含s'这个状态未来可能的价值。这里的q节点相当于一个chance节点

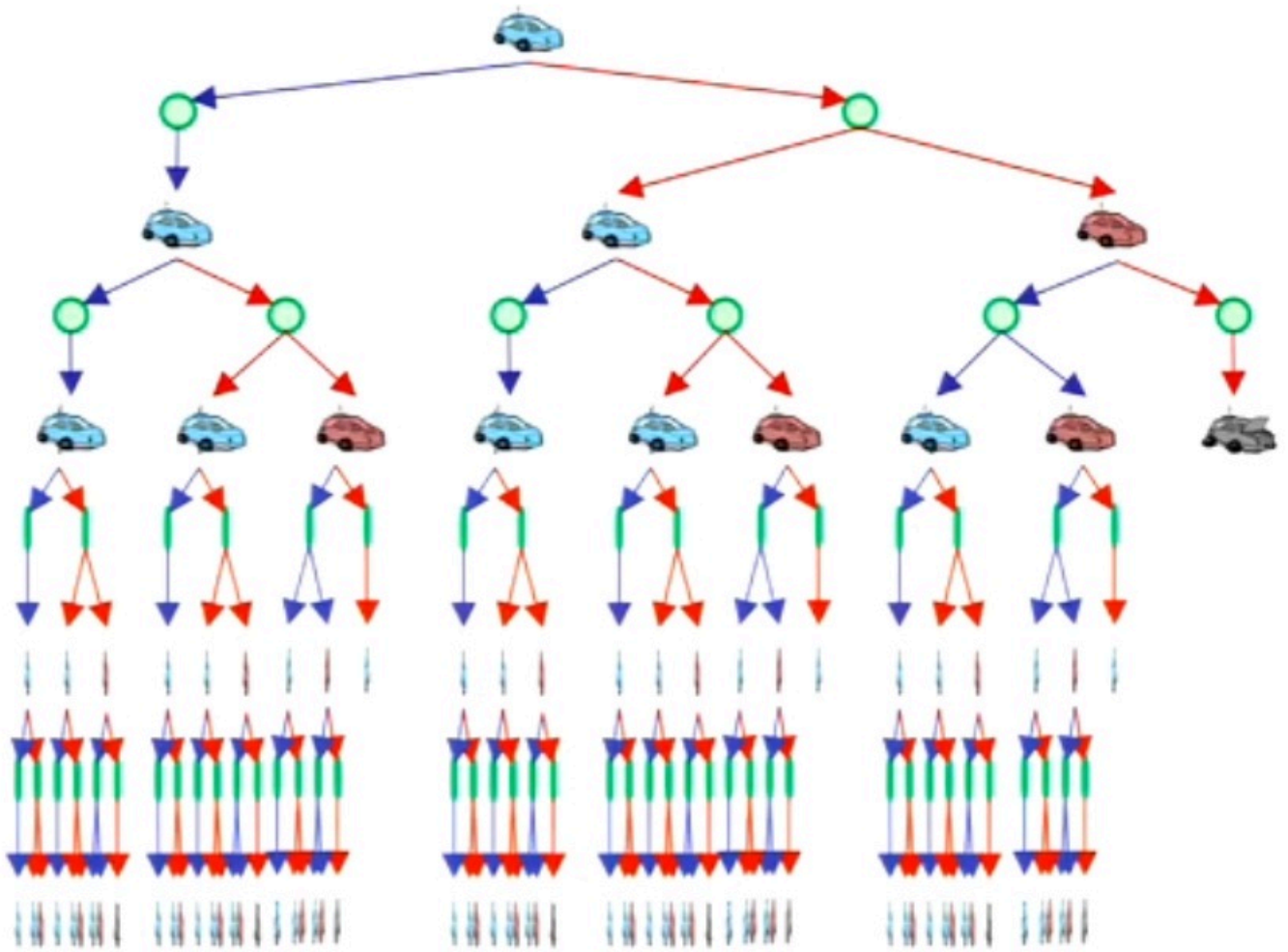
, , , , ,

我们可以联立两个式子，建立一个方程，把某个状态的 V^* 与其他状态的 V^* 联系起来，这个式子叫**贝尔曼方程**

, , , , ,

- 效用值 utility （折扣后）奖励的总和
 - 价值 value 从一个状态开始未来的预期效用值
 - q-value 从一个q状态开始未来的的预期效用值
- 上面只是用递推的方法重新给出了价值的具体定义

价值迭代算法 Value Iteration

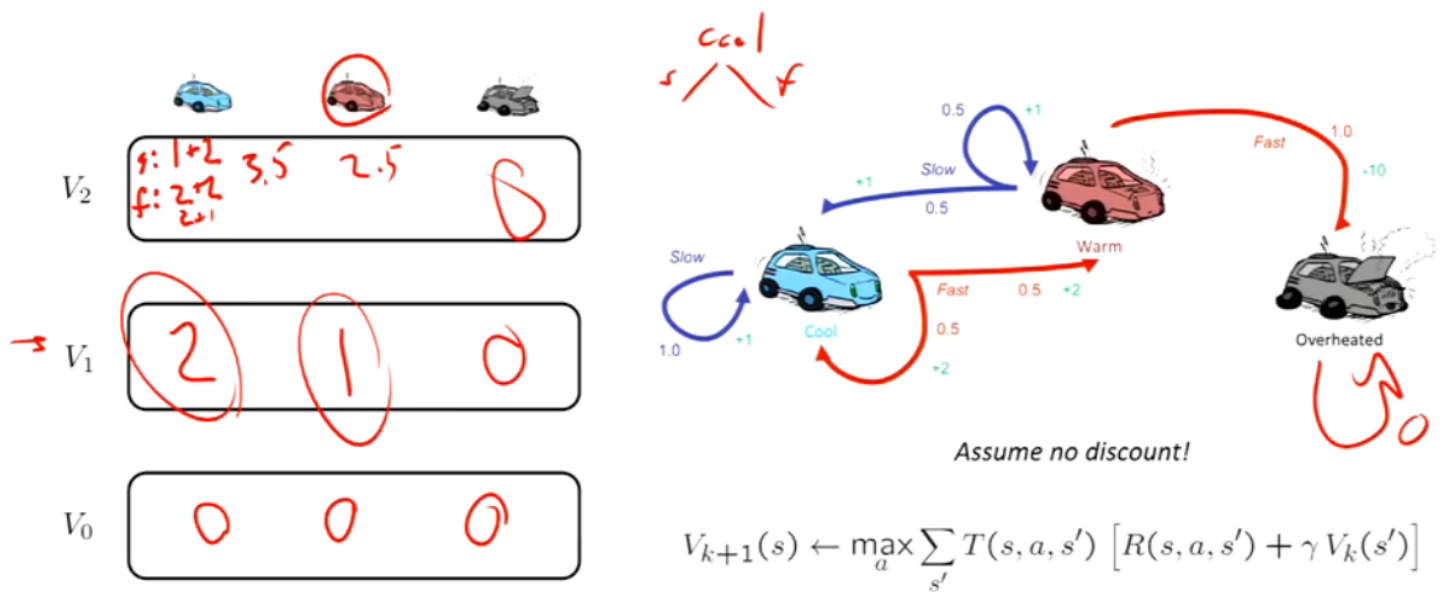


我们可以使用期望最大化算法，但是有两个明显问题：

- 这个树是无限的（虽然可以用深度受限的方法截断）
- 树实际上只有三个状态，有很多子树是完全相同的，但是在重复出现（而且 γ 小于1时，过深的部分影响很小）

价值迭代算法通过记录限制深度的子树的价值 $V_k(s)$ ，解决子树重复的问题。其中的 k 是一个时间步数，代表从这个状态开始，还能做几次行动与期望量。初始时 $V_0(s) = 0$ ，接着开始考虑 $V_1(s)$ ，代表这个状态做一次行动可以得到的最大价值，然后 $V_2(s)$ 其实过程就类似深度受限的 i ，但是这个过程是可以复用迭代的。比如算 $V_2(s)$ 时无需一直到根节点再返回，进行一次操作后，就相当于 $V_1(s)$ ，所以根据贝尔曼方程：
$$V_{k+1}(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$
 因而计算每一个 $V_{k+1}(s)$ ，其实就是深度为1的 expectimax

举个例子：



总共有三种状态：Cool Warm Overheated

过热状态只有一种行动，所以 $V_0(\text{Overheated}) = V_1(\text{Overheated}) = V_2(\text{Overheated}) = 0$

当 $k=0$ ，三种状态的 V_0 都为 0，因为无法采取行动

因此 $V_0(\text{Cool}) = V_0(\text{Warm}) = 0$

当 $k=1$ ，Cool 和 Warm 状态可以有两种 action，slow ($R=1$) 或者 fast ($R=2$) -> 这一步相当于计算 $\max_a$

对于 Cool 状态，两种 action 中，fast 能得到的最大 $R=2$

对于 Warm 状态，两种 action，fast 会导致过热 ($R=-10$)，故选择 slow ($R=1$)

因此 $V_1(\text{Cool}) = 2$; $V_1(\text{Warm}) = 1$

重点是 $k=2$ 的时候！

对于 Cool 状态

如果选择 slow，这一步得到 +1 奖励，还剩一步就相当于 $V_1(\text{cool}) = 2$ ，因而选择 slow 的总价值为 $1+2=3$

如果选择 fast，这一步得到 +2 奖励，还剩一步有 0.5 的概率转移到 $V_1(\text{cool}) = 2$ ，有 0.5 概率转移到 $V_1(\text{warm}) = 1$ ，因此选择 fast 的总价值为 $2 + 0.5 \times 2 + 0.5 \times 1 = 3.5$

-> 这一步是计算 $\sum_{s'} T(s, a, s')$ 之前没有出现过这一项，而且状态可以复用！不需要一路算到 V_0

因此 $V_2(\text{Cool}) = 3.5$ -> $\max_{\text{action}}$

同理可以得到 $V_2(\text{Warm}) = 2.5$

这个例子比较特殊，折扣因子是 1，所以结果不会收敛；而且奖励函数是在状态转移时生效的

当迭代一定次数后，只要 $\gamma < 1$ ， v_k 就会收敛到 v^*

复杂度是 $O(s^2 a)$

策略评估和策略提取

为了评估不同策略的优劣，定义一个固定的策略下的效用值：

$$V^{\pi}, \pi, \pi, \pi$$

遵守一个特定的策略 π 时， V 是相似的，只不过没有了 $\max_a$ ，因为 a 由 $\pi(s)$ 决定，这实际上已经是一个线性方程

策略评估 Policy Evaluation

有了上面固定策略下的效用值迭代式子，就可以通过类似价值迭代的方法得到某个 π

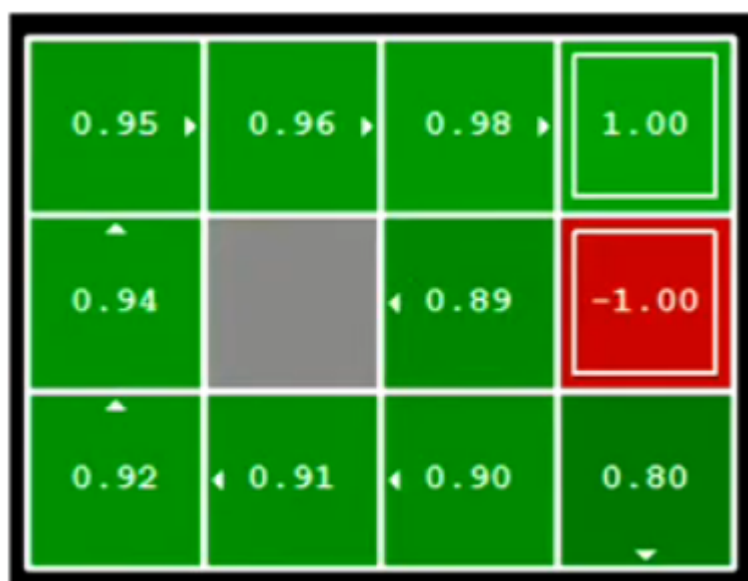
令

$$\pi_1, \pi, \pi, \pi$$

因为少了对所有action遍历求最大，复杂度是 $O(s^2)$

策略提取 Policy Extraction

如果已经知道各个状态下的 V^* ，该怎么得到最优的策略？



这并不是显然的向着value大的地方移动就行，而要遍历所有可能的action，得到哪个action能得到最大的 V ：

$$\pi, , , ,$$

(argmax 返回取到最大值的参数 a)

只不过因为各个状态下的 V^* 已知，公式里的 $V^*(s')$ 是已知的，相当于一步expectimax
特别的，如果知道是 Q^* ，那么可以直接知道策略，因为 $qstate$ 就是action后的价值

策略迭代算法 Policy Iteration

价值迭代存在以下问题（建议看BV1HcqpYwEw6 55min处的讲解）：

- $O(S^2 A)$ 的复杂度太高，尤其是State多的时候
- 往往在价值收敛之前，策略就已经得到了收敛

所以给出策略迭代算法，该算法重复策略评估和策略提取两步，直到策略收敛：

$$\pi_{i1}, \pi_i, \pi_i, \pi_i$$

上式进行到V收敛为止

$$\pi_{i1}, \dots, \pi_i$$

上式根据第一步给出的value提取出最优的策略

- 最优的
- 特定条件下收敛快很多

总结

上面提到的内容其实都是贝尔曼方程的变体，也都是一层的expectimax（迭代，V_k已经得知），区别只是遍历了action还是有个确定的策略